Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамента цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины «Искусственный интеллект и машинное обучение»
Вариант 12

Выполнила:
Коробка В.А.,
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль) «Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Руководитель практики:
Воронкин Роман Александрович, доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: Работа с Jupyter Notebook, JupyterLab и Google Colab

Цель: исследовать базовые возможности интерактивных оболочек Jupyter Notebook, JupyterLab и Google Colab для языка программирования Python.

Порядок выполнения работы

Был выполнен пример 1 из лабораторной работы. Была импортирована библиотека Matplotlib для построения графиков и включено отображение графиков непосредственно в Jupyter Notebook с помощью команды %matplotlib inline. Создана последовательность значений x и список y, содержащий квадраты этих значений. После этого функция plt.plot() была использована для построения графика. В результате отображается график квадратичной функции.

```
from matplotlib import pylab as plt
%matplotlib inline
```

```
x = [i for i in range(50)]
y = [i**2 for i in range(50)]
plt.plot(x, y)
```

```
[<matplotlib.lines.Line2D at 0x1b4857e5310>]
```

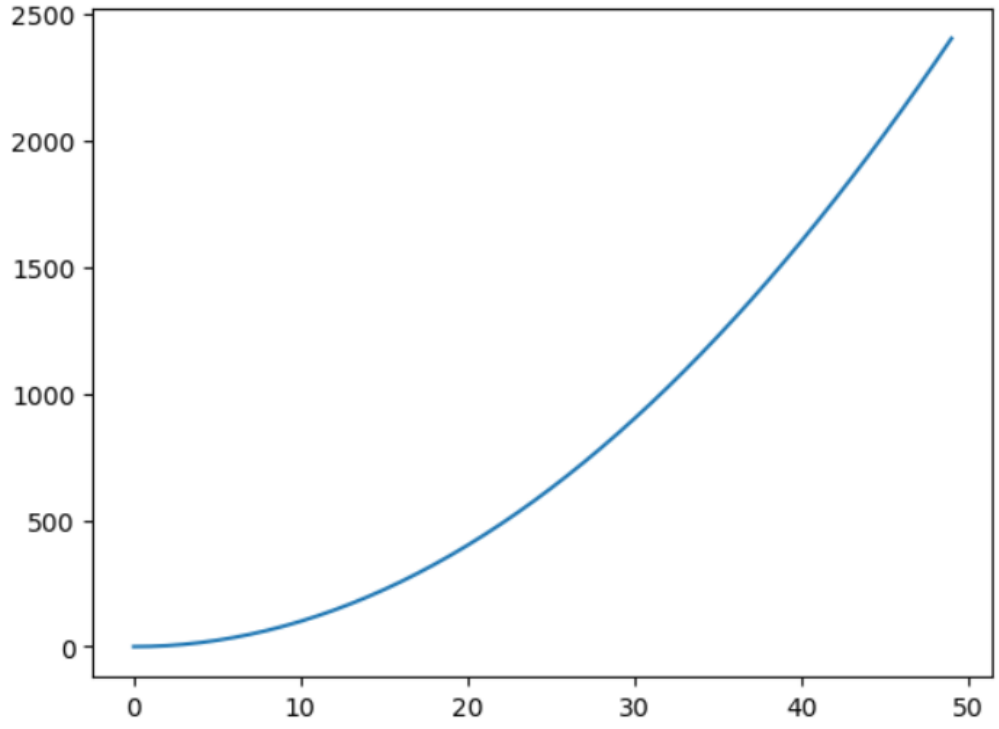

Рисунок 1 – Пример 1 из лабораторной работы

Был выполнен пример 2 из лабораторной работы. В нем используются заголовки разных уровней, различные стили оформления текста: полужирный шрифт, курсив и выделение фрагмента как встроенного кода. Показано создание маркированного списка для структурирования информации. В конце приведён пример вставки математической формулы.

Заголовок первого уровня

Заголовок второго уровня

Полужирный текст, *курсив*, `код в строке`

Список:

- Пункт 1
- Пункт 2
- Пункт 3

Формула: $y = mx + b$

[ ]:

Рисунок 2 – Пример 2 из лабораторной работы

Был выполнен пример 3 из лабораторной работы. Импортируется модуль os. Функция os.getcwd() используется для получения текущей рабочей директории, в которой выполняется программа. В результате выполнения на экран выводится путь C:\Users\nout1, показывающий активную папку.

[1]:

```
import os
print(os.getcwd())
```

```
C:\Users\nout1
```

Рисунок 3 – Пример 3 из лабораторной работы

Был выполнен пример 4 из лабораторной работы. Используются библиотеки NumPy и Matplotlib для построения графика функции. С помощью функции np.linspace() создаётся массив значений (x) на интервале от 0 до 10. Затем вычисляются значения функции y = sin(x). Команды plt.plot(), plt.xlabel(), plt.ylabel() и plt.title() строят график и добавляют подписи осей и заголовок. В результате выполнения отображается график синусоиды.

```
[1]: import matplotlib.pyplot as plt
     import numpy as np

[2]: x = np.linspace(0, 10, 100)
     y = np.sin(x)

[3]: plt.plot(x, y)
     plt.xlabel("X")
     plt.ylabel("Y")
     plt.title("Графики синусоиды")
     plt.show()
```

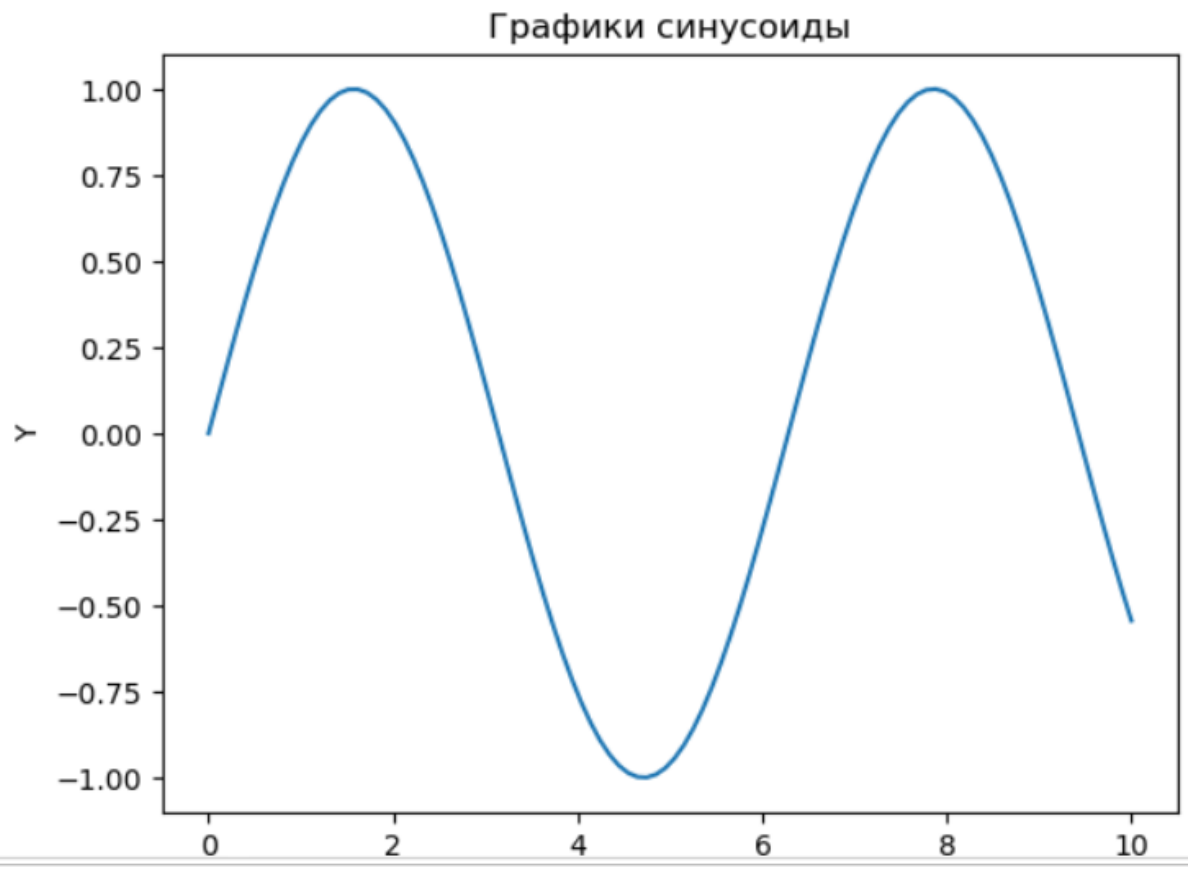

Рисунок 4 – Пример 4 из лабораторной работы

Было выполнено индивидуальное задание по варианту 12. Условие задания:

Создайте новую Markdown-ячейку и:

Напишите заголовок "Практическое задание "1" .

Добавьте жирный и курсивный текст.

Создайте нумерованный и маркированный списки.

Вставьте согласно индивидуального задания, например:

Вставьте изображение через ![Описание](URL) .

Определение обратной матрицы (если детерминант ненулевой)

Используются заголовки различных уровней, оформление текста полужирным и курсивным шрифтом, создание нумерованных и маркированных списков. Приведено математическое определение обратной матрицы с использованием формул. В конце добавлено изображение, вставленное по ссылке.

Практическое задание 1

Форматирование текста

Тут **жирный текст** Тут *курсивный*

Нумерованный список

1. Шоколад
2. Мороженое
3. Лимонад

Маркированный список

- Брокколи
- Фасоль
- Картошка

Определение обратной матрицы

Матрица A^{-1} называется обратной к квадратной матрице A, если

$$A \cdot A^{-1} = A^{-1} \cdot A = E$$

где E — единичная матрица. Обратная матрица существует, если

$$\det(A) \neq 0.$$

Изображение

Рисунок 5 – Индивидуальное задание 1

Было выполнено задание 1.2. Условие задания:

Запросите у пользователя его имя с помощью input().

Выведите приветствие: "Привет, <имя>! Добро пожаловать в JupyterLab /Google Colab!"

Запустите ячейку (Shift + Enter).

Программа запрашивает у пользователя ввод имени с помощью функции input(). Введённое значение сохраняется в переменной name. Затем с использованием форматированной строки выводится персонализированное приветствие.

```
[1]: name = input("Введите ваше имя: ")
     print(f"Привет, {name}! Добро пожаловать в JupyterNotebook!")

     Введите ваше имя:  Лера
     Привет, Лера! Добро пожаловать в JupyterNotebook!
```

Рисунок 6 – Выполнение задания 1.2

Было выполнено задание 2. Условие задания:

Создайте и сохраните текстовый файл с помощью open().

Запишите в него несколько строк текста.

Закройте файл и затем откройте его снова, считав содержимое и выведя на экран.

Проверьте, существует ли файл, используя os.path.exists().

Удалите файл с помощью модуля os.

Сначала создаётся текстовый файл task2.txt, в который записывается несколько строк текста с помощью функции open() в режиме записи. Затем файл открывается в режиме чтения, его содержимое считывается и выводится на экран. С помощью функции os.path.exists() проверяется существование файла, после чего файл удаляется с использованием функции os.remove(). В результате программа подтверждает наличие файла и его успешное удаление.

```
[1]: import os

with open("task2.txt", "w") as f:
    f.write("Один\n")
    f.write("Два\n")
    f.write("Три\n")
    f.write("Четыре\n")

with open("task2.txt", "r") as f:
    content = f.read()
    print("Содержимое файла:\n", content)

print("Файл существует:", os.path.exists("task2.txt"))

os.remove("task2.txt")
print("Файл удален!")
```

```
Содержимое файла:
 Один
Два
Три
Четыре

Файл существует: True
Файл удален!
```

Рисунок 7 – Выполнение задания 2

Было выполнено задание 3. Условие задания:

Выведите список всех доступных магических команд (%lsmagic).

Используйте %time и %timeit для измерения времени выполнения кода.

Создайте Python-скрипт в Jupyter (%%writefile script.py) и выполните его через !python script.py

Выведите список файлов в текущей директории с помощью %1s .

Используйте %history для просмотра истории команд.

Команда %lsmagic выводит список доступных магических команд, которые могут применяться для работы с кодом и средой выполнения. С помощью %time

и %timeit измеряется время выполнения программы. Команда %%writefile используется для создания Python-скрипта прямо из ноутбука, после чего скрипт запускается через команду !python script.py. Команда %history отображает историю ранее выполненных команд.

```
[1]: %lsmagic

[1]: root
       line
       cell

[2]: %ls

     Том в устройстве C имеет метку Windows
     Серийный номер тома: 725F-CA3B

     Содержимое папки C:\Users\nout1

    14.04.2026  18:34    <DIR>          .
    16.04.2025  05:59    <DIR>          ..
    12.04.2026  22:10    <DIR>          .anaconda
    13.04.2026  17:37             4 287 .bash_history
    14.04.2026  18:35    <DIR>          .conda
    17.08.2025  18:04    <DIR>          .config
```

Рисунок 8 – Выполнение задания 3

```
[3]: %time sum(range(1000000))
     %timeit sum(range(1000000))

     CPU times: total: 62.5 ms
     Wall time: 65.2 ms
     66.6 ms ± 27.5 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

[6]: %%writefile script.py
     print("Скрипт успешно запущен!")
     for i in range(5):
         print("Число:", i)

     Overwriting script.py

[7]: !python script.py

     Скрипт успешно запущен!
     Число: 0
     Число: 1
     Число: 2
     Число: 3
     Число: 4

[8]: %history

     %lsmagic
     %ls
     %time sum(range(1000000))
     %timeit sum(range(1000000))
     %writefile script.py
     print("Скрипт успешно запущен!")
     for i in range(5):
         print("Число:", i)
     !python script.py
     %%writefile script.py
     print("Скрипт успешно запущен!")
     for i in range(5):
```

Рисунок 9 – Выполнение задания 3

Было выполнено задание 4. Условие задания:

Выведите список файлов в текущей директории с помощью !ls .

Проверьте, какой Python используется (!which python).

Создайте папку test_folder (!mkdir test_folder) и убедитесь, что она появилась.

Переместите файл в новую папку и удалите его.

Очистите вывод в ячейке (!clear).

Рисунок 10 – Выполнение задания 4

Было выполнено задание 5. Условие задания:

Подключите Google Drive к Colab с помощью команды:

from google.colab import drive

drive.mount('/content/drive')

Проверьте, что ваш диск успешно подключился, используя !ls /content/drive/MyDrive

Создайте и сохраните текстовый файл в Google Drive:

Откройте файл my_text_file.txt в режиме записи.

Запишите в него несколько строк текста.

Закройте файл и убедитесь, что он появился в папке MyDrive.

Прочитайте файл из Google Drive:

Откройте ранее созданный файл.

Считайте его содержимое и выведите в ячейке.

Создайте и сохраните CSV-файл вручную, используя Microsoft Excell или Libre Office Calc:

Создайте список, в котором будут строки с данными о студентах (например, ФИО, возраст, группа).

Запишите этот список в CSV-файл в Google Drive вручную, используя стандартные методы записи в файл.

```
from google.colab import drive

drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!ls /content/drive/MyDrive
```

```
'Colab Notebooks'   fruits.csv   my_text_file.txt
```

```
file_path = "/content/drive/MyDrive/my_text_file.txt"
with open(file_path, "w") as f:
  f.write("Это мой тестовый файл в Google Drive\n")
  f.write("Это мой любимый тестовый файл в Google Drive\n")
print("Файл сохранен в Google Drive")
```

```
Файл сохранен в Google Drive
```

```
with open(file_path, "r") as f:
  content = f.read()
  print("Содержимое файла: \n", content)
```

```
Содержимое файла:
 Это мой тестовый файл в Google Drive
Это мой любимый тестовый файл в Google Drive
```

```
students = [
    ["Иванов И.И.", 20, "ИВТ-101"],
    ["Петров П.П.", 22, "ИВТ-102"],
    ["Сидорова А.А.", 21, "ИВТ-103"],
]

csv_path = "/content/drive/MyDrive/students.csv"
with open(csv_path, "w") as f:
  for student in students:
    f.write(",".join(map(str, student)) + "\n")

print("Файл students.csv  сохранен в Google Drive.")
```

```
Файл students.csv  сохранен в Google Drive.
```

Рисунок 11 – Выполнение задания 5

Было выполнено дополнительное задание. Условие задания:

Дополнительно: попробуйте загрузить файл из Google Drive в Colab с помощью files.upload() , обработать его и сохранить обратно.

Была выполнена загрузка CSV-файла в Google Colab с помощью функции files.upload(). После загрузки файл students.csv открывается, считывается с использованием модуля csv, содержимое выводится на экран. В список студентов добавляется новая запись. В конце обновлённые данные записываются обратно в файл students.csv.

```
from google.colab import files
uploaded = files.upload()
```

```
Выбрать файлы students.csv
students.csv(text/csv) - 106 bytes, last modified: 16.04.2026 - 100% done
Saving students.csv to students.csv
```

```
import csv
input_file = "students.csv"
with open(input_file, "r", encoding="utf-8") as f:
  reader = csv.reader(f)
  data = list(reader)
print("Содержимое файла:")
for row in data:
  print(row)
```

```
Содержимое файла:
['Иванов И.И.', '20', 'ИВТ-101']
['Петров П.П.', '22', 'ИВТ-102']
['Сидорова А.А.', '21', 'ИВТ-103']
```

```
data.append(["Коробка В.А.", 19, "ИВТ-б-о-24-1"])
for row in data:
  print(row)
```

```
['Иванов И.И.', '20', 'ИВТ-101']
['Петров П.П.', '22', 'ИВТ-102']
['Сидорова А.А.', '21', 'ИВТ-103']
['Коробка В.А.', 19, 'ИВТ-б-о-24-1']
```

```
output_path = "students.csv"
with open(output_path, "w", newline="", encoding="utf-8") as f:
  writer = csv.writer(f)
  writer.writerows(data)
print("Файл сохранён обратно в students.csv!")
```

```
Файл сохранён обратно в students.csv!
```

Рисунок 12 – Выполнение дополнительного задания

Ответы на контрольные вопросы

1. Какие основные отличия JupyterLab от Jupyter Notebook?

JupyterLab - это более современная среда с вкладками, файловым менеджером и терминалом. Jupyter Notebook - более простой интерфейс, работающий с одним документом за раз.

2. Как создать новую рабочую среду (ноутбук) в JupyterLab?

В JupyterLab нужно нажать Launcher → Notebook → выбрать ядро (Python) или через File → New → Notebook.

3. Какие типы ячеек поддерживаются в JupyterLab и как их переключать?

Поддерживаются ячейки Code, Markdown и Raw. Тип ячейки меняется через выпадающий список сверху или клавишами Esc → M (Markdown), Y (Code).

4. Как выполнить код в ячейке и какие горячие клавиши для этого используются?

Код выполняется кнопкой Run или клавишами Shift + Enter. Также используются Ctrl + Enter (выполнить без перехода) и Alt + Enter (выполнить и создать новую ячейку).

5. Как запустить терминал или текстовый редактор внутри JupyterLab?

В Launcher можно выбрать Terminal или Text File. Также через меню File - New - Terminal/Text File.

6. Какие инструменты JupyterLab позволяют работать с файлами и структурами каталогов?

JupyterLab имеет встроенный File Browser, позволяющий открывать, создавать, удалять и перемещать файлы и папки.

7. Как можно управлять ядрами (kernels) в JupyterLab?

Ядрами можно управлять через меню Kernel: перезапускать, останавливать или менять ядро (Restart, Change Kernel).

8. Каковы основные возможности системы вкладок и окон в интерфейсе JupyterLab?

Интерфейс поддерживает вкладки, разделение окон и перетаскивание файлов, позволяя работать с несколькими документами одновременно.

9. Какие магические команды можно использовать в JupyterLab для измерения времени выполнения кода? Приведите примеры.

Для измерения времени используются магические команды %time и %timeit. Например: %time sum(range(1000)) или %timeit x**2.

10. Какие магические команды позволяют запускать код на других языках программирования в JupyterLab?

Используются команды типа %%bash, %%javascript, %%html, позволяющие запускать код на других языках внутри ячейки.

11. Какие основные отличия Google Colab от JupyterLab?

Google Colab работает в облаке и не требует установки. JupyterLab запускается локально на компьютере пользователя.

12. Как создать новый ноутбук в Google Colab?

Нужно открыть сайт Google Colab и выбрать File - New notebook.

13. Какие типы ячеек доступны в Google Colab, и как их переключать?

В Colab доступны Code и Text (Markdown) ячейки. Переключение выполняется через меню или кнопки + Code и + Text.

14. Как выполнить код в ячейке Google Colab и какие горячие клавиши для этого используются?

Код выполняется кнопкой или клавишами Shift + Enter. Также работают Ctrl + Enter и Alt + Enter.

15. Какие способы загрузки и сохранения файлов поддерживает Google Colab?

Colab поддерживает загрузку с компьютера, Google Drive, GitHub и скачивание файлов обратно на устройство.

16. Как можно подключить Google Drive к Google Colab и работать с файлами?

Google Drive подключается командой:

from google.colab import drive

drive.mount('/content/drive').

После подключения файлы Drive становятся доступны как обычные папки.

17. Какие команды используются для загрузки файлов в Google Colab из локального компьютера?

Для загрузки файлов используется модуль files: from google.colab import files - files.upload(). После выполнения появляется окно выбора файла с компьютера.

18. Как посмотреть список файлов, хранящихся в среде Google Colab?

Список файлов можно посмотреть командой !ls или через файловую панель слева в интерфейсе Colab.

19. Какие магические команды можно использовать в Google Colab для измерения времени выполнения кода? Приведите примеры.

Используются магические команды %time и %timeit. Например: %time sum(range(1000)) или %timeit x**2.

20. Как можно изменить аппаратные ресурсы в Google Colab (например, переключиться на GPU)?

Нужно открыть Runtime - Change runtime type и выбрать аппаратный ускоритель (GPU или TPU), затем сохранить настройки.

Вывод: В ходе лабораторной работы были изучены основные возможности сред Jupyter Notebook, JupyterLab и Google Colab, включая работу с ячейками, Markdown, файлами и магическими командами. Получены навыки выполнения Python-кода, визуализации данных, работы с файловой системой и облачным хранилищем Google Drive.